TransitOps - Step-by-Step Implementation Plan

Objective: Build TransitOps in a structured, scalable manner using React (Vite), Express.js, TypeScript, Neon PostgreSQL, Drizzle ORM, Vercel, and Render.

Phase 1 - Project Setup

- Initialize Git repository
- Setup React + Vite frontend
- Setup Express + TypeScript backend
- Configure ESLint, Prettier
- Create folder structure
- Configure environment variables
- Connect GitHub repository

Deliverable: Working frontend & backend boilerplates connected to GitHub.

Phase 2 - Database Setup

- Provision Neon PostgreSQL
- Configure Drizzle ORM
- Create schema files
- Generate migrations
- Seed roles and demo users
- Verify database connection

Deliverable: Database schema, migrations, seeded data.

Phase 3 - Authentication

- Implement JWT Authentication
- Hash passwords using bcrypt
- Create login endpoint
- Store JWT in HTTP-only cookies
- Protect routes
- Implement RBAC middleware

Deliverable: Secure authentication system with role-based authorization.

Phase 4 - Core UI

- Create dashboard layout
- Sidebar & Navbar
- Theme setup
- Reusable components
- Protected routes
- Responsive layouts

Deliverable: Professional SaaS dashboard shell.

Phase 5 - Vehicle Module

- Vehicle CRUD
- Validation
- Search
- Filters
- Status badges

Deliverable: Complete vehicle management.

Phase 6 - Driver Module

• Driver CRUD
• License validation
• Safety score
• Status management

Deliverable: Driver management with compliance checks.

Phase 7 - Trip Management

• Trip creation
• Vehicle & Driver assignment
• Business rule validation
• Dispatch
• Complete
• Cancel

Deliverable: Complete trip lifecycle.

Phase 8 - Maintenance & Expenses

• Maintenance workflow
• Fuel logs
• Expense logs
• Automatic status updates

Deliverable: Operational tracking modules.

Phase 9 - Dashboard & Analytics

• KPI Cards
• Charts
• Fleet utilization
• ROI
• CSV Export

Deliverable: Interactive analytics dashboard.

Phase 10 - Testing

• Unit testing
• API testing
• Validation testing
• Business rule verification
• Responsive testing

Deliverable: Stable application with verified workflows.

Phase 11 - Deployment

• Deploy backend to Render
• Deploy frontend to Vercel
• Configure environment variables
• Connect Neon database
• Production testing

Deliverable: Publicly accessible production deployment.

Phase 12 - Final Polish

• Loading skeletons
• Toast notifications
• Empty states
• Error states
• Performance optimization
• README & documentation

Deliverable: Hackathon-ready polished application.

Recommended Build Sequence

1. Project Setup
2. Database & Drizzle Schema
3. Authentication & RBAC
4. Dashboard Layout
5. Vehicle Management
6. Driver Management
7. Trip Management
8. Maintenance Module
9. Fuel & Expense Module
10. Analytics Dashboard
11. Testing
12. Deployment
13. Final UI Polish

Deployment Checklist

• Frontend deployed on Vercel
• Backend deployed on Render
• Neon PostgreSQL connected
• Environment variables configured
• HTTPS enabled
• CORS configured
• Production API verified
• Final smoke testing completed